Assignment 4: Dependency Extraction

December 2, 2025

Website : https://softwarearchitecture.wixsite.com/eecs4314assignment

Group Name: SourceArchitects

Team Members Information

Name	Student ID	Email
Sheida Khodabakhshian Khansari	215534340	sheidakh@my.yorku.ca
Vrinda Ashwar	218624692	vrinda03@my.yorku.ca
Matthew Lee	218256743	mattl7@my.yorku.ca
Werner Bagu-Paiz	21727772	wernerbp@my.yorku.ca
William Moran	218928960	willvm24@my.yorku.ca
Stefewn Johnson	219411099	stef9@my.yorku.ca

1. Abstract **3**
2. Introduction **3**
3. Dependency Extraction Techniques **3**
3.1. Understand 3
3.2. Include 4
3.3. srcML 4
4. Quantitative Analysis **5**
4.1. Comparison Process 5
4.2. Understand Vs. Include 5
4.3. Include Vs. srcML 6
4.4. Understand Vs. srcML 7
4.5. Understand Vs. Vs. Include Vs. srcML 7
4.6. Implications and Metrics(Precision and Recall) 8
4.7. Random Sampling 9
4.8. Understand and Include Precision and Recall 10
4.9. Include and srcML Precision and Recall 10
4.10. Understand and srcML Precision and Recall 11
5. Qualitative Analysis **11**
5.1. Understand 11
5.2. Include 12
5.3. srcML 12
6. Risks and Limitations **12**
7. Use Cases/Sequence Diagrams **13**
8. Lessons Learned **15**
9. Conclusion **16**
10. References **16**

1. Abstract

This report compares different techniques for extracting dependencies from the OpenCV codebase. First is the Understand, then an include based extractor extractor, and finally a srcML based structural extractor. Each method captures different forms of information and the choice of technique affects the accuracy and completeness of the resulting dependency graph. Quantitative and qualitative analyses were conducted to evaluate the overall reliability of each approach. The findings highlight the strengths and limitations of extraction strategies and demonstrate how combining multiple techniques can lead to a more comprehensive understanding of large software architectures.

2. Introduction

Learning software architecture dependency is an essential process in the analysis and maintenance of large software systems. Dependencies in complex, modular codebases like OpenCV can help identify interactions between components and the way functionality is shared among subsystems, and structural coupling can pose a maintenance risk. Since various dependency extraction methods retrieve different kinds of information, including raw syntactic relations to deep semantic relations, the choice of method has a strong influence on the quality and completeness of the architectural view generated. This report will compare and contrast three dependency extraction methods: the semantic analysis generated by Understand, a custom extractor based on include implemented in Python, and a custom srcML-based extractor that extracts dependencies based on XML views of source code. The assignment is not simply aimed at creating these dependencies, but also to critically examine the differences in how each technique varies in its behaviour, precision, and interpretability in the context of applying it to the OpenCV system.

The report then describes the implementation and design process of the two new extraction methods, their justification of the workflow, and, where necessary, examples in the form of use cases or sequence diagrams. It then compares all three methods on quantitative measures, like the total dependency counts, intersections, and the pairwise differences, and then their qualitative comparison occurs using statistically sampled pairs of dependencies. Comparison metrics such as precision, recall are talked about to determine how reliable a particular method is. Lastly, the report evaluates the overall implications of the findings, risks, and limitations of the two methods of extraction, and then summarizes the lessons learned during the process. This report conducted a comparative analysis of the syntactic, structural, and semantic extraction strategies to evaluate the various dependency extraction methods. It did this in terms of how they each build architectural representations, where each method suits best, where it underperforms, and ways in which they can be combined to provide a more reliable architectural analysis.

3. Dependency Extraction Techniques

3.1. Understand

Understand is a commercial, static analysis environment that can create the entire abstract syntax tree (AST) of the whole OpenCV codebase [1]. It is the first architectural dependency extraction tool applied to the OpenCV source code 4.12.0, intending to compare how various static analysis tools expose architectural structures. Using Understand can help to resolve overloaded functional dependencies, namespaces, include paths, etc [1]. Understand processes its analysis outputs into TA form as facts of the form

$INSTANCE, which are the source files and relational form as facts of the form “cLinks”, which are the relationships like file inclusion, function calls, type references, and macro dependencies. The output of the TA file is used as the ground truth to compare with the two custom extraction methods used as part of the assignment.

3.2. Include

The second method that we used to help extract the dependencies of OpenCV derives dependencies directly out of C/C++ #include directives. This approach is specifically light and does not look at semantic behaviour, but instead considers only syntactic inclusion relationships. We created a Python program that recursively searches the OpenCV directory tree, finds all the .cpp, .hpp, .h, and .inl files, and searches each file with a regular expression to extract all the include statements. Every file found is listed in the form of an $INSTANCE element of the TA output file called Include_Extraction.ta. Whenever there is a #include statement found, the Python program that we created attempts to resolve the included filename to an actual file path within the OpenCV tree by matching basename occurrences. On finding a successful match, an edge is added between the including file and the included file with a “cLinks”. Sometimes, no matches are found; the relation is still included, but marked with the raw included target instead. This method identifies structural dependencies using the inclusion of a header and thus generates a dependency graph that is generally smaller and easier than the one generated by Understand. Although using the “include” directives has its advantages of low implementation and complexity, and reproducibility, it fails to model semantic relationships like the use of types or calling functions, and so only a portion of the actual dependency structure of the system is extracted.

3.3. srcML

The third method that our group decided to use is the srcML tool, which is used to convert C/C++ source code to an XML format, revealing syntactic and syntax structure elements such as <cpp: include>, <function>,<call>, and <decl> elements [2]. First, we needed to download srcML to be called in another Python code. The second Python program was written to read each source file by running srcML to produce XML, interpret the XML tree to find the definition of functions and call expressions, and map the individual function call to the file where the same name is defined. Then, just like the include method, each file is recorded as an $INSTANCE, while each resolved function-call relationship is output as a “cLinks” edge, forming a detailed dependency graph. Theoretically, srcML has more semantic information than the include-based method since it includes behaviour-level connectivity. But in practice, and particularly with a large and complicated project such as OpenCV, srcML generated fewer dependencies than the include-based method. This would happen for reasons such as that it doesn’t cover unused/commented dependencies, and a lot of OpenCV elements that utilize the more advanced C++ functionality that is beyond the capabilities of srcML to resolve in its XML. However, the srcML method still offers useful behaviour and understanding, and integrates with the strictly structured include-based dependencies and the overall semantic dependencies provided by Understand.

4. Quantitative Analysis

4.1. Comparison Process

The raw TA file extracted from Understand was converted to the proper format that could be compared to the dependency files extracted using include method and srcML. This was done using a python script called “convert.py” which cleans a raw TA file by removing all occurrences of the string “opencv-4.12.0/”. Dependencies were then compared using a custom python script called “comparison.py” .This Python script performs a pairwise comparison of dependencies listed in three .ta files that include the extracted dependencies from Understand, Include and srcML . It first reads each file and extracts all dependency names, ignoring empty lines or comments. Then, for every pair of dependency sets, it finds the common dependencies as well as those exclusive to each set. The script prints the number of common and exclusive dependencies for each pairwise comparison. It also saves a detailed report to “pairwise_comparison.txt”, where the names of the dependencies common or exclusive to each set in each pairwise comparison is saved . Overall, it provides a clear view of overlap and differences between dependency files extracted using the three different methods.

4.2. Understand Vs. Include

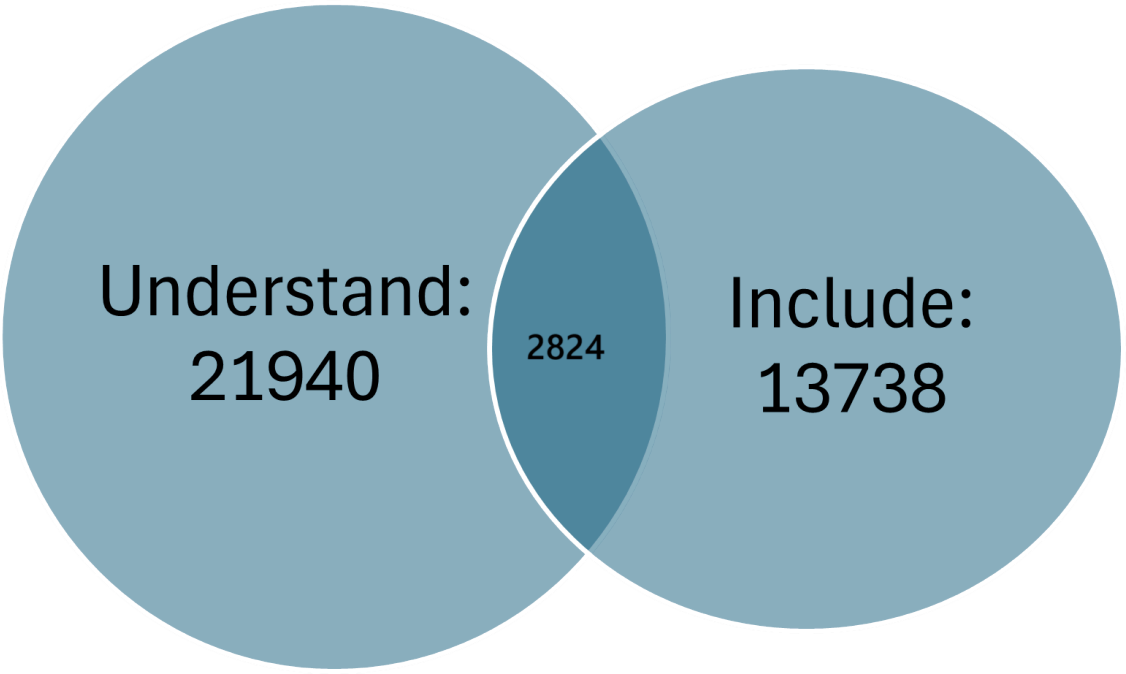
Common dependencies: 2824

total dependencies for Understand: 24764

total dependencies for Include: 16562

Exclusive to Understand: 21940

Exclusive to Include: 13738



As shown in the figure above, Understand was able to extract more dependencies than the Include method. The number of dependencies it found was 24764 dependencies. This includes the 2824 dependencies that were also discovered by the Include method as well as 21940 other dependencies not discovered by Include. In total it was able to discover 24764/ (13738+24764)=24764/38502=0.64=64% of the total dependencies extracted by these two techniques. Meanwhile, the Include methodology was able to extract 13738 more dependencies but along with the common dependencies with Understand, it was only able to extract (13738 +2824)/38502 = 0.43 = 43% of the total dependencies extracted by these two techniques. The overlap coverage or the percentage of common dependencies found between these two techniques is 2824/38502 = 0.073 = 7.3% of the total dependencies extracted by these two methods.

4.3. Include Vs. srcML

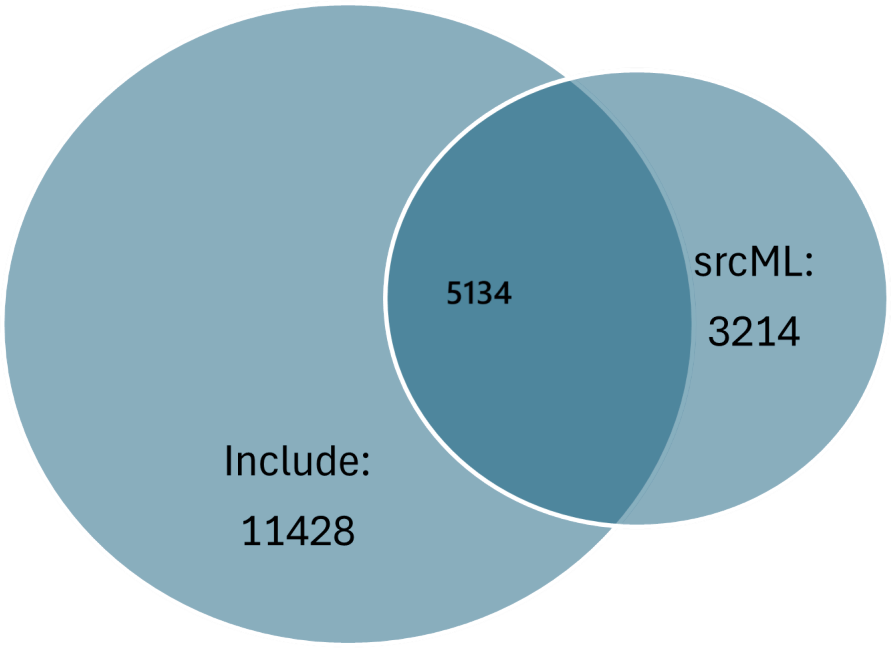
Common dependencies: 5134

total dependencies for Include: 16562

total dependencies for srcML: 8348

Exclusive to Include: 11428

Exclusive to srcML: 3214



As shown in the figure above, the Include methodology was able to extract more dependencies than srcML methodology. The technique that used srcML was only able to extract (3214+5134)/(3214+5134+11428) = 0.422 = 42.2% of the total dependencies extracted by these two techniques. Meanwhile, Include was able to extract (11428+5134)/(3214+5134+11428)=0.837=83.7% of the total dependencies extracted by these two techniques.The overlap coverage is 5134/(3214+5134+11428) = 0.26 = 26%

4.4. Understand Vs. srcML

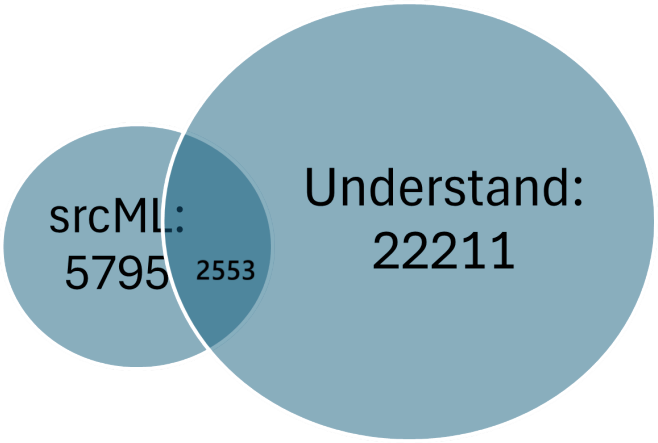
Common dependencies: 2553

total dependencies for Understand: 24764

total dependencies for srcML: 8348

Exclusive to Understand: 22211

Exclusive to srcML: 5795



As shown in the figure above, Understand was able to extract more dependencies than the technique that utilized srcML. The number of dependencies it found was 24764 dependencies. This includes the 2553 dependencies that were also discovered by the other technique as well as 22211 other dependencies not discovered by srcML methodology. In total it was able to discover 24764 /(24764 +5795) = 81% of the total dependencies extracted by these two techniques. Meanwhile, the srcML methodology was able to extract 5795 more dependencies but along with the common dependencies with Understand, it was only able to extract (5795+2553)/(24764 +5795)=0.27=27% of the total dependencies extracted by these two techniques. The overlap coverage is 2553/(24764 +5795) = 0.08 = 8%

4.5. Understand Vs. Vs. Include Vs. srcML

Common to all three: 1928

Common Understand & Include: 896

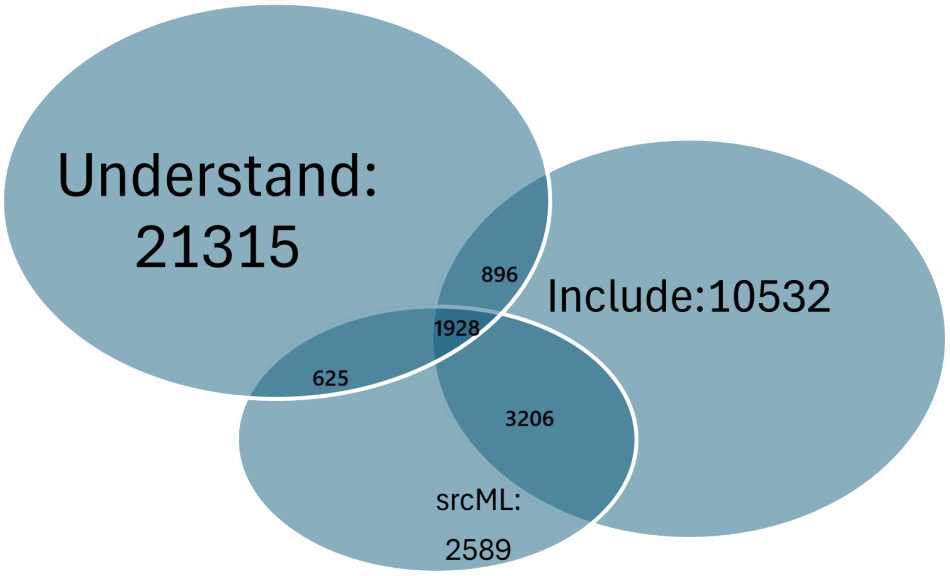
Common Understand & srcML: 625

Common Include & srcML: 3206

Exclusive to Understand: 21315

Exclusive to Include: 10532

Exclusive to srcML: 2589



As shown in the figure, Understand is the most effective tool in extracting dependencies followed by the Include methodology and then the srcML utilized technique. Understand was able to find (21315+ 896+625+1928) /(21315+ 896+625+1928+10532+2589+3206)= 24764/41091=0.60 = 60% of the total dependencies extracted by these three methods while Include methodology was only able to find (10532+1928+896+3206) /41091= 0.40 = 40% of the total dependencies. The srcML methodology was able to find (1928+625+3206+2589) /41091=0.20 = 20%. The overlap coverage is 1928/41091 = 0.0469 = 4.69%.

The three dependency extraction techniques differ in the types of dependency relationships they can detect. Understand performs semantic analysis and can detect various dependency types including function-call relationships, class inheritance, type usage, template instantiation, variable and constant references, macro-expanded includes, and fully resolved #include links. Meanwhile the Include method is syntactic and can only detect literal include statements in source files. These include statements may include unresolved, redundant, or inactive ones as well. Meanwhile, srcML can be used for structural level dependency extraction. It can convert the source code into XML allowing for the syntactic search of elements such as <cpp:include>. However, it only detects structurally recognized include statements not including inactive/commented or macro-generated ones leading it to extract less dependencies. Overall, Understand detects more dependencies than the other two techniques, however it is not able to extract all dependencies.

4.6. Implications and Metrics(Precision and Recall)

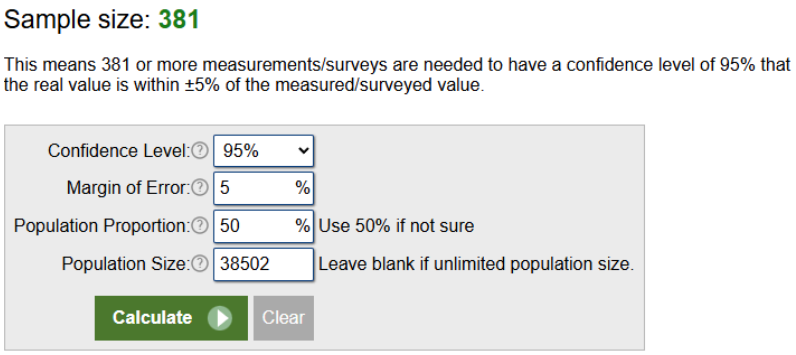
Performance metrics like precision and recall are very important and are widely used [3]. Its main uses are to assess the performance of the information retrieval and classification systems [3]. The meaning of precision is the proportion of retrieved instances which are relevant [3]. In simple words, the number of items that are labelled positive by the system is correct [3]. Now recall's meaning is the proportion of all items that are relevant, that the system is able to retrieve successfully [3]. It reflects how complete the identification is [3]. Now these two metrics, precision and recall, emphasise a tradeoff [3]. When you improve the precision, recall actually goes down, and the same is for the other way [3]. When recall goes up, precision goes down [3]. The reason this occurs is because lowering false positives means

missing some of the positives that are true, while attempting to catch all relevant instances, which, as a result, may bring in more irrelevant instances [3]. Being able to understand recall, and precision is very important, especially when we assess dependency extraction techniques [3]. The reason is that it gives us insights into how reliable a technique is able to catch a real dependency [3]. This comes from the recall metric [3]. Another reason is that it tells us the number of dependencies that are actually valid [3]. This comes from the precision metric [3].

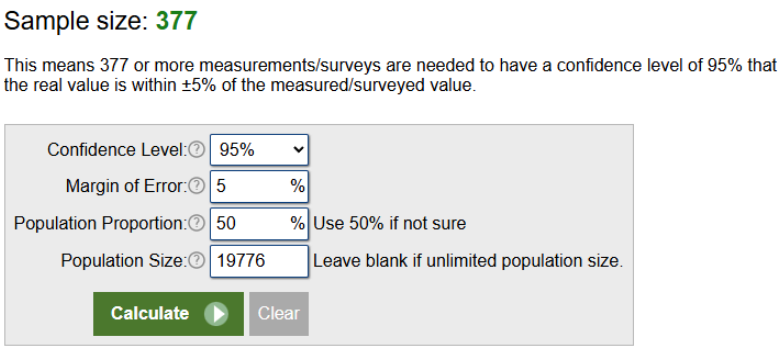
4.7. Random Sampling

For each respective pair we will use a random sampling size of 380 to have a confidence level of 95% with a ± 5% margin. The population size is the total number of dependencies for each pair (common + exclusive dependencies for each method). We will run a python script make_samples.py to randomly take 380 total edges from each of the pairwise text files. The edges can either be common or exclusive to their respective extraction method. Each pair of methods will be outputted to a sample_380.txt file storing random edges from those dependencies to do our analysis on.

Understand and Include (21940 + 2824 + 13738 = 38502)



Include and srcML(11428 + 5134 + 3214 = 19776)



Understand and srcML (5795 + 2553 + 22211 = 30559)

Sample size: **380**

This means 380 or more measurements/surveys are needed to have a confidence level of 95% that the real value is within ±5% of the measured/surveyed value.

Confidence Level:	95%	
Margin of Error:	5 %	
Population Proportion:	50 %	Use 50% if not sure
Population Size:	30559	Leave blank if unlimited population size.

Calculate Clear

We will be performing three pairwise comparisons among the three extractors (Understand/Include, Understand/srcML and Include/srcML). For each pair, we evaluated both tools by swapping the roles of reference and evaluated extractor, resulting in six precision/recall measurements in total. The precision and recall values per tool will be averaged to obtain an overall score for each extraction technique.

We will be using these formulas to calculate the precision and recall of each pair.

Precision=True Positives/(True Positives+False Positives)
Recall=True Positives/(True Positives+False Negatives)

4.8. Understand and Include Precision and Recall

Understand (Reference) Vs. Include (Evaluated)

TP = 28, FN = 222, FP = 130

Precision for Include = 28/(28+130) = 0.177

Recall for Include = 28/(28+222) = 0.112

Include (Reference) Vs. Understand (Evaluated)

TP = 28, FN = 130, FP = 222

Precision for Understand = 28/(28+222) = 0.112

Recall for Understand = 28/(28+130) = 0.177

4.9. Include and srcML Precision and Recall

Include (Reference) Vs. srcML (Evaluated)

TP = 92, FN = 217, FP = 71

Precision for srcML = 92/(92+71) = 0.564

Recall for srcML = 92/(92+217) = 0.298

srcML (Reference) Vs. Include (Evaluated)

TP = 92, FN = 71, FP = 217

Precision for Include = 92/(92+217) = 0.298

Recall for Include = 92/(92+71) = 0.564

4.10. Understand and srcML Precision and Recall

Understand (Reference) Vs. srcML (Evaluated)

TP = 35, FN = 276, FP = 69

Precision for srcML = 35/(35+69) = 0.337

Recall for srcML = 35/(35+276) = 0.113

srcML (Reference) Vs. Understand (Evaluated)

TP = 35, FN = 69, FP = 276

Precision for Understand = 35/(35+276) = 0.113

Recall for Understand = 35/(35+69) = 0.337

Extraction Method	Average Precision	Average Recall
Understand	(0.112 + 0.113)/2 = 0.1125 or 11.25%	(0.177 + 0.337)/2 = 0.257 or 25.7%
Include	(0.177 + 0.298)/2 = 0.2375 or 23.75%	(0.112 + 0.564)/2 = 0.338 or 33.8%
srcML	(0.564 + 0.337)/2 = 0.4505 or 45.05%	(0.298 + 0.113)/2 = 0.2055 or 20.55%

5. Qualitative Analysis

5.1. Understand

Sample modules from Understand:
modules/core/src/matrix.cpp
modules/imgproc/src/floodfill.cpp
modules/dnn/src/layers/…
modules/calib3d/src/…
modules/features2d/src/…

Understand only edges come from real semantic relationships inside of OpenCV such as function calls, inline utilities and template instantiations across modules. These dependencies do not appear in raw #include statements which is why Include and srcML fail to capture them. Sample modules such as core, imgproc, dnn, calib3d and features2d show that Understand captures these deeper architecture links. The average precision is 11.25% which is low however it is this way because the other extraction methods miss these real dependencies. Its recall being 25.7% shows that Understand does capture many meaningful relationships.

5.2. Include

Sample modules from Include:
3rdparty/openexr/…
3rdparty/libjasper/...
3rdparty/libjpeg…
samples/cpp/…
modules/world/src/precomp.hpp

Include only edges come from the raw #include statements which consist of things like long chains of indirect headers, system and STL files, 3rd party libraries and sample/test code. These dependencies technically exist but do not reflect OpenCV's logical architecture resulting in a lot of noise. Include has an average precision of 23.75% and recall of 33.8% meaning Include is very broad and captures a lot of edges, many being irrelevant from an architectural standpoint.

5.3. srcML

Sample modules from srcML:
3rdparty/openexr/…
3rdparty/zlib-ng/…
3rdparty/libwebp/…
hal/carotene/src/...

srcML only edges are mostly vendor libraries and backend optimization code such as openexr, zlib-ng, libwebp and carotene hal. Since srcML operates at the Abstract Syntax Tree (AST) level it identifies structural relationships and type declarations but cannot resolve semantic relationships like Understand [6]. The fact that it misses information such as cross-module function calls, template instantiations and inline dependencies means its output is incomplete. With the highest average precision of 45.05% and lowest recall of 20.55%, srcML doesn't produce as much noise as Include, but it misses most of the deeper architectural dependencies in the codebase.

6. Risks and Limitations

The dependency extraction results in this assignment are influenced by several risks and limitations that come from the tools used, the nature of OpenCV's codebase [4], and the comparison process itself. These limitations affect the completeness, accuracy, and interpretability of the architectural conclusions drawn from the three techniques.

Understand provides the most comprehensive output among the three techniques, but treating it as the reference creates an inherent risk. Although Understand performs deep semantic analysis using compiler-level information and fully resolves symbol usage, includes, and type references [1], it can still misinterpret dependencies in areas where OpenCV [4] relies on templates, macro-generated code, and architecture-specific dispatch. Its output further depends on the compilation configuration (macros, include paths, platform flags), meaning different builds may produce different dependency sets. Therefore, even though Understand yields the largest and most detailed graph, it cannot be assumed to represent the exact architecture in all environments.

The Include-based extraction method is limited by its strictly syntactic nature. It records literal #include statements without determining whether the included file affects actual behaviour or architecture. This leads to numerous false positives, especially from sample files, test code, vendor libraries, and system headers present inside OpenCV's repository [4]. The method resolves include targets using filename matching, which risks incorrectly linking files with identical basenames across multiple OpenCV modules [4]. Because the approach does not evaluate semantic usage, macro logic, or conditional compilation, many meaningful dependencies are either missed or only appear under specific build conditions. As a result, Include-based relationships tend to over-approximate structural dependencies and do not reflect runtime or algorithm-level interactions.

The srcML extraction technique introduces another set of limitations. Although srcML exposes structural elements such as <call> and <include> through its XML representation [2], it does not perform semantic resolution. It cannot correctly link overloaded functions, template instantiations, inline utilities, or cross-module references; Features that are heavily used within OpenCV's imgproc and core modules [4]. This leads to many missed dependencies and explains why srcML produced fewer edges overall compared to Include and Understand. In practice, srcML often captured dependencies from 3rd-party or backend code included within the OpenCV source tree [4] because these files use simpler syntax patterns that srcML can easily parse. Additionally, running srcML across a large codebase like OpenCV [4] can be time-consuming and memory-intensive, increasing the chance of incomplete or skipped extractions.

There are also limitations in the comparison methodology. All extracted outputs were normalized into the TA format, which removes dependency type information (e.g., include vs call vs type reference). This flattening limits the ability to interpret whether dependencies represent architectural coupling or incidental structural references. Precision and recall metrics were calculated by treating Understand as the reference; however, because Understand itself may contain configuration-dependent or over-resolved relationships, these metrics must be interpreted cautiously. Precision and recall inherently emphasize different trade-offs in classification accuracy [3], and relying on a non-perfect reference skews both values. The qualitative analysis used a sample size of 380 dependencies, which reduces manual effort but introduces sampling uncertainty; some modules or dependency categories may not be fully represented. Finally, OpenCV contains extensive 3rd-party vendor code [4], and these files often introduce noise in Include and srcML-based results, inflating or distorting the perceived differences between techniques.

Overall, the combination of tool limitations, syntactic vs semantic differences, and comparison assumptions means that none of the methods provides a complete or perfectly accurate architectural view of OpenCV [4]. Understand may over-resolve, Include may over-approximate, and srcML may under-approximate dependencies.

7. Use Cases/Sequence Diagrams

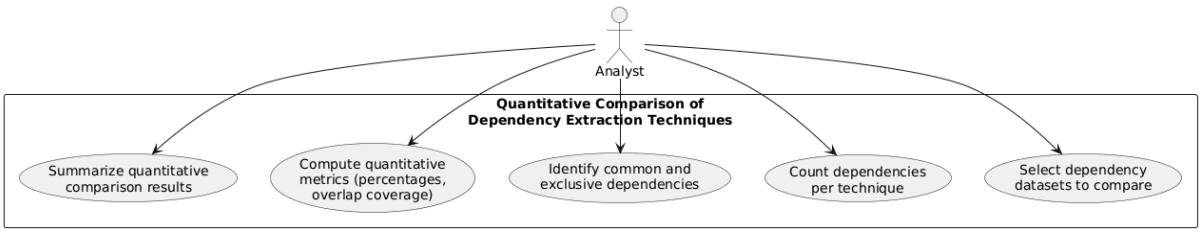


Figure 1 [5]: Use Case Diagram for Dependency Comparison Workflow

The use case diagram shows the workflow of comparing dependency extraction techniques used by the analyst. It shows that they compute quantitative metrics, identify common and exclusive dependencies, select datasets, and summarize results. The diagram shows the analyst interacting with the system at a high level.

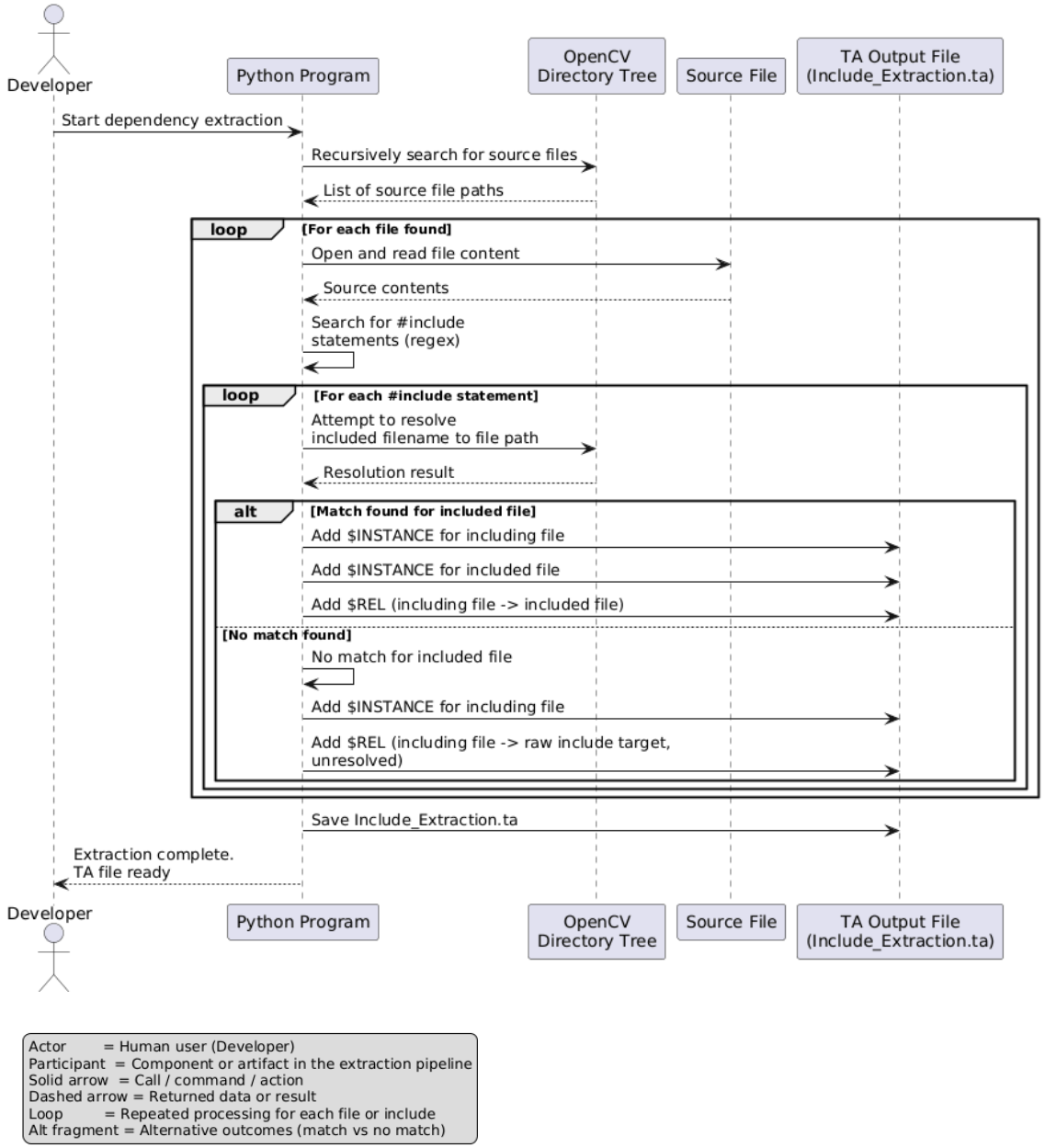


Figure 2 [5]: Sequence Diagram for Include Based Extraction Process

This sequence diagram shows the execution of the custom Python script used for the include based dependency extraction. It visualizes how the developer's program recursively scans the OpenCV directory tree, opens each file, searches for include statements, attempts to resolve each referenced file, and records the results in TA format. The diagram shows the difference between matched and unmatched include targets.

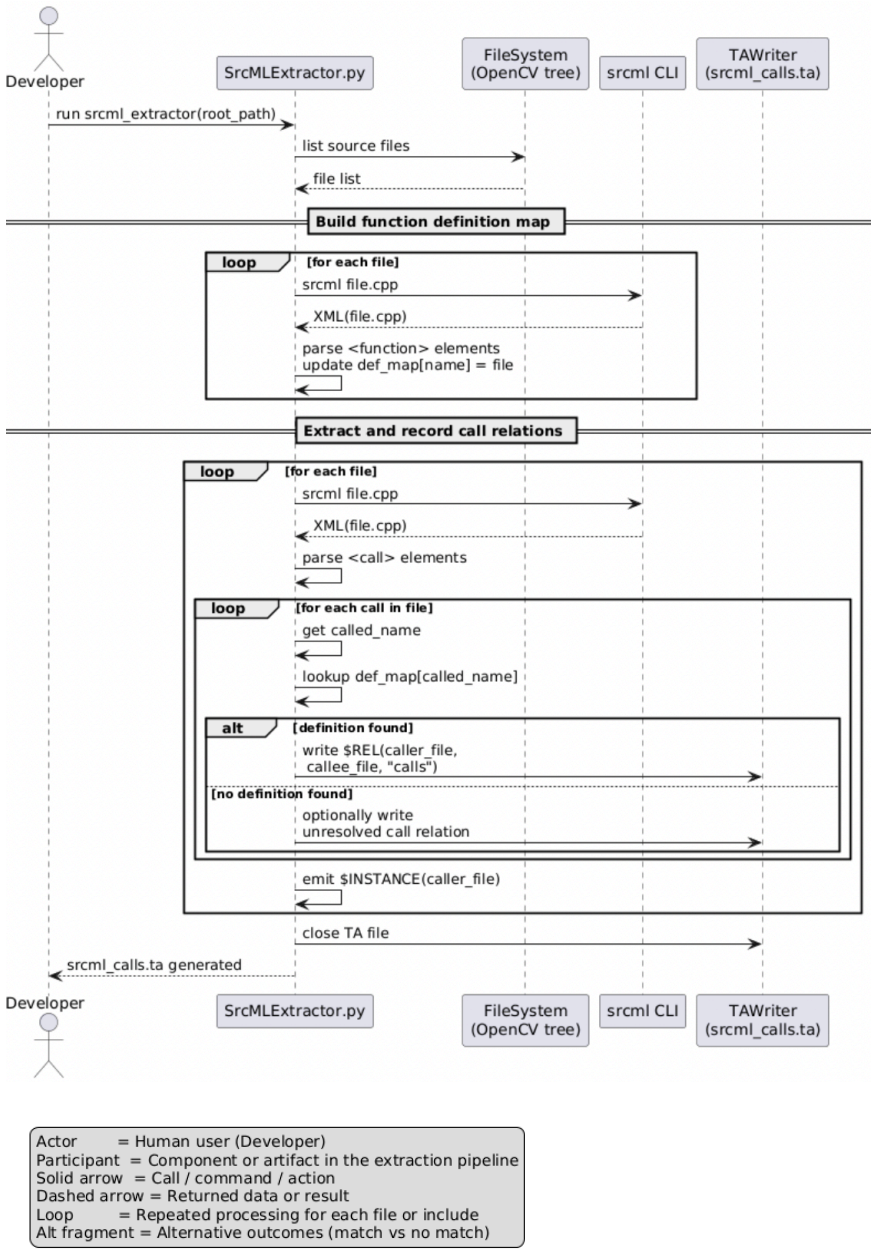


Figure 3 [5]: Sequence Diagram for srcML Based Extraction Process

The srcML extraction diagram shows files that are processed by converting them into XML and interpreting function and call elements. It illustrates two main phases. The first is figuring out where functions are defined, and the second is checking which files call those functions. srcML turns each file into XML, the tool identifies where functions are defined, and then connects each function call to its definition. The diagram shows resolved vs. unresolved calls, demonstrating how TA entries are generated.

8. Lessons Learned

Some things we learned are that dependency extraction tools are very different when it comes to the coverage and the detail they give. We noticed that Understand gives the most semantic information. It also found the highest number of dependencies that are meaningful. Include, on the other hand, is very simple and quick. However, it catches literal #include

statements only. This would result in limits in its architectural accuracy. We also have srcML, which extracts structural information like element relationships and function calls. However, it misses many dependencies because of its lack of semantic resolution. Comparison metrics like precision and recall give us a very useful way to assess how reliable each of the tool's extracted results is. In conclusion, picking the right tool and combining it with quantitative metrics and qualitative sampling is very important for meaningful architectural insights in big projects.

9. Conclusion

This comparison showed that each dependency extraction technique provides a different level of insight into the OpenCV architecture. Understand delivered the most complete results, while the include extractor offered broad but included many unnecessary dependencies. srcML produced cleaner outputs but skipped many dependencies since it can't analyze deeper semantics. The quantitative and qualitative analyses confirmed that no single method is fully accurate on its own. Overall, combining semantic, syntactic, and structural techniques gives a more reliable and balanced understanding of larger systems.

10. References

[1] "Understand: The Software Developer's Multi-Tool," *Scitools.com*, 2024. https://scitools.com/

[2] "srcML," *Srcml.org*, 2025. https://www.srcml.org/ (accessed Nov. 19, 2025).

[3] "Precision and Recall," Accessed: Nov. 22, 2025. [Online]. Available: https://www.britannica.com/science/precision-and-recall

[4] "OpenCV: Open Source Computer Vision Library," GitHub Repository. https://github.com/opencv/opencv

[5] "PlantUML," PlantUML.com, https://plantuml.com/ (accessed Oct. 28, 2025).

[6] "srcML – Wikipedia." Accessed: Nov. 25, 2025. [Online]. Available: https://en.wikipedia.org/wiki/SrcML